

Stackademic

How to Implement The Supertrend Trading Strategy in Python for a Potential Good Return

This article guides you on coding and backtesting the Supertrend indicator in Python. It's a strategy that uses price action and volatility to identify trend direction.



Tradie Capital

Follow

14 min read · Mar 31, 2024

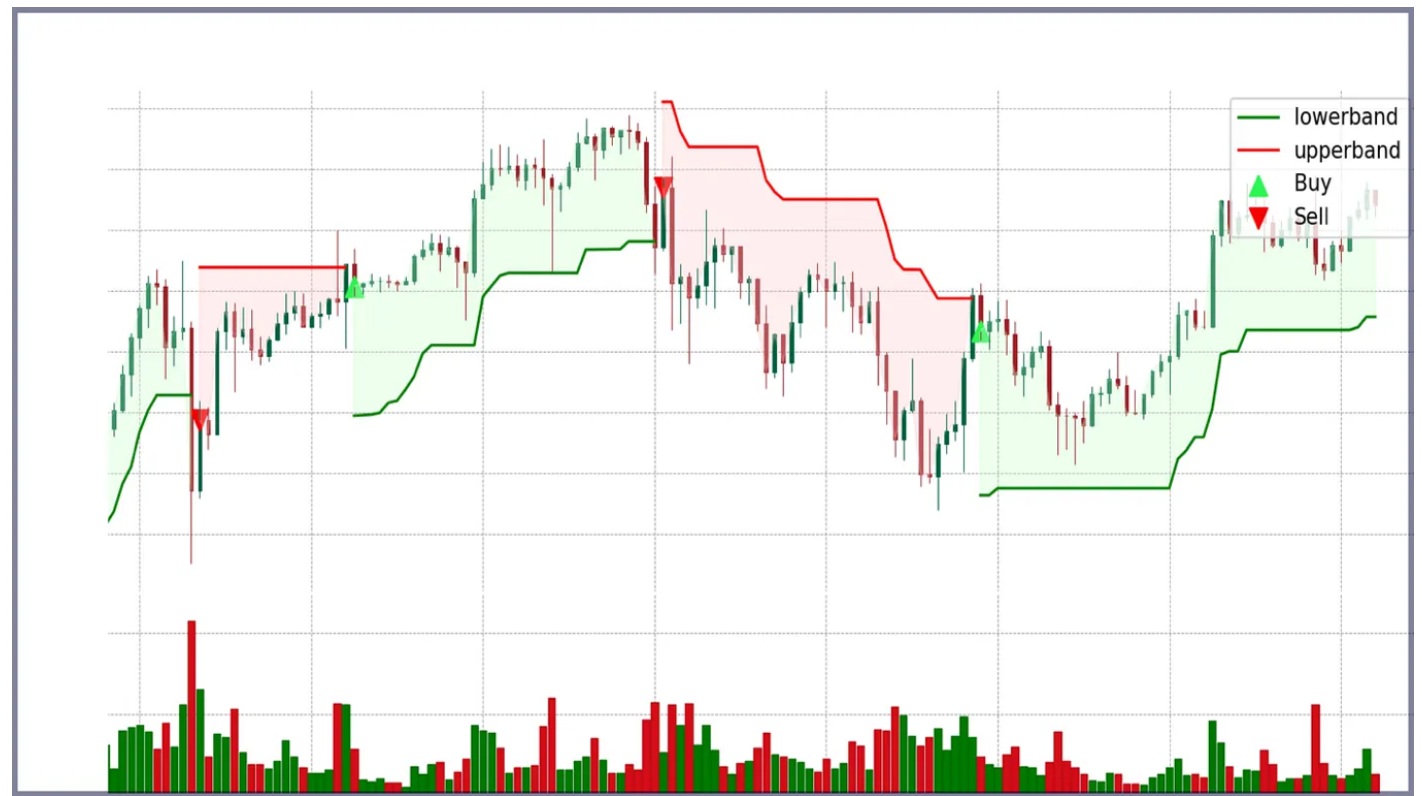


479



5





Extract from Strategy output

Introduction

It is a wonderful morning and you're sitting there browsing through your TradingView chart configuration looking for some hot signals, only to be confronted with uncertain trends. You seem to want to bend space-time — but, it feels like you're just trying to fold the fabric of reality with your mind. It might be frustrating when you want to make a move but the

market isn't your regular friend at the grocery store — it just isn't cooperating! Trust me, we've all been there — it sucks. However, you cannot force the market to behave in your favor. Timing is pretty much everything, you only can wait until the stars align, I guess. We simply have to wait for the perfect moment. That's when the Supertrend indicator comes in handy! It's a tool that kind of takes an analysis of the specified target markets and watches out for trends and reversals. All you need is to make good investment decisions, of course, some can blow out on you, but it's a game of consistency — you need more wins than losses. You need to know when to enter or exit the market.

Why would you want to use Python for this implementation?

If you have worked with Python before, you know that it is a very versatile and comprehensive programming language with great libraries, specially designed for data analysis and visualization. Pandas is one of such libraries that provides efficient ways to import, clean, and preprocess market data, making it ready for analysis. It is also superb at handling large datasets, an essential aspect when dealing with market data.

Talk of Numpy another useful library for financial analysis. It is very

powerful in providing an extensive collection of mathematical functions and tools that come in handy with statistical calculations. These functions speed up computations over large datasets, such as when dealing with market data, thereby providing deeper insights and analysis in a short amount of time. This, compared to manually looking through numerous charts, can be quite time-consuming and may not lead to accurate analysis.

I bet you don't want to trade with bias and emotional sentiments that may lead to bad investment decisions. This is why you may want to consider an algorithmic trading strategy such as this one that can potentially lead to higher returns.

How does the Supertrend indicator work?

Let's picture this, a river sometimes flows smoothly and other times behaves so rough and may break the river banks, this is pretty much similar to the markets — prices can move smoothly or swing wildly and spike through certain unexpected ranges. So the supertrend indicator takes a look at these swings, which is known as volatility. It does this by measuring the Average True Range (ATR); which is basically the trading

range of an asset within a specified period. The supertrend then uses the ATR to generate the upper band and the lower band — which are more like boundaries within which the asset prices range within that period. If the price breaks the upper boundary that signals an uptrend and a break through the lower band signals a downward trend. This is a straightforward concept — right?

Calculating the Supertrend

First and foremost, we calculate the Average True Range(ATR). This is the measure of volatility. It is calculated over a specified period of time.

You want to start by getting the True Range(TR) for each data point. A data point can be collected per day, 4-hour timeframe, hourly, or by minute, etc. It all depends on the timeframe setup.

$$TR = \text{Max}(\text{High} - \text{Low}, \text{Abs}(\text{High} - \text{Prev Close}), \text{Abs}(\text{Low} - \text{Prev Close}))$$

We then calculate the ATR for the past specified period, say we could do a period of 14. So this means that the ATR is calculated for every 14 data points(take these as rows in the dataframe) — which is pretty much the

average of the True Ranges(TR) within that period.

$$\left(\frac{1}{n}\right) \sum_i^n \text{TR}_i$$

where:

TR_i = Particular true range, such as first day's TR,
then second, then third

n = Number of periods

Pyhton Implementation

Having understood the core concepts of the supertrend strategy, we now have to get hands-on and execute it with Python.

Step 1: Prepare your coding Environment

Install the latest Python 3 distribution if you don't have it already. You can check the version of Python on your machine by running “*python -v*” in

your terminal.

Install all the required Python libraries such as Pandas, Numpy, Matplotlib, mplfinance, and ccxt.

```
pip install ccxt pandas matplotlib pandas-ta mplfinance
```

Step 2: Import Dependencies

```
import ccxt
import warnings
from matplotlib.pyplot import fill_between
import pandas as pd
import numpy as np
import pandas_ta as ta
import mplfinance as mpf
import matplotlib.pyplot as plt
warnings.filterwarnings('ignore')
```

Import the required dependencies for the project. The package ccxt is the

data provider that connects to several crypto exchanges — in this case we will use it to connect to Binance. The other libraries Pandas and Numpy are for statistical computations while mplfinance and matplotlib are for creating plots to visualize the data.

Step 3: Fetch the asset data from a provider(ccxt)

```
def fetch_asset_data(symbol, start_date, interval, exchange):  
    # Convert start_date to milliseconds timestamp  
    start_date_ms = exchange.parse8601(start_date)  
    ohlcv = exchange.fetch_ohlcv(symbol, interval, since=start_date_ms)  
    header = ["date", "open", "high", "low", "close", "volume"]  
    df = pd.DataFrame(ohlcv, columns=header)  
    df['date'] = pd.to_datetime(df['date'], unit='ms')  
    df.set_index("date", inplace=True)  
    # Drop the last row containing live data  
    df.drop(df.index[-1], inplace=True)  
    return df
```

The function “*fetch_asset_data*” is getting the historical price data needed for backtesting. It takes in 4 arguments— the ticker symbol, start date, time period (like daily, hourly etc.), and the exchange the trading pair is on.

First it changes the start date into a number that the exchange API understands. Then it connects to that exchange and requests for all the open, high, low, close prices (OHLC) from that start date until now for the ticker symbol.

It then puts all that price data into a neat dataframe and drops the last row of data. Why? Because that last row might have live prices data that are changing right now. And we can't use future prices when we're backtesting what happened in the past! So it drops that last row to give us only historical data before today. Then it returns the clean dataframe table to us.

Step 4: Calculate the Supertrend lines

```
def supertrend(df, atr_multiplier=3):  
    # Calculate the Upper Band(UB) and the Lower Band(LB)  
    # Formular: Supertrend =(High+Low)/2 + (Multiplier)*(ATR)  
    current_average_high_low = (df['high']+df['low'])/2  
    df['atr'] = ta.atr(df['high'], df['low'], df['close'], period=15)  
    df.dropna(inplace=True)  
    df['basicUpperband'] = current_average_high_low + (atr_multiplier * df['a  
    df['basicLowerband'] = current_average_high_low - (atr_multiplier * df['a  
    first_upperBand_value = df['basicUpperband'].iloc[0]  
    first_lowerBand_value = df['basicLowerband'].iloc[0]
```

```

upperBand = [first_upperBand_value]
lowerBand = [first_lowerBand_value]

for i in range(1, len(df)):
    if df['basicUpperband'].iloc[i] < upperBand[i-1] or df['close'].iloc[i] > upperBand[i-1]:
        upperBand.append(df['basicUpperband'].iloc[i])
    else:
        upperBand.append(upperBand[i-1])

    if df['basicLowerband'].iloc[i] > lowerBand[i-1] or df['close'].iloc[i] < lowerBand[i-1]:
        lowerBand.append(df['basicLowerband'].iloc[i])
    else:
        lowerBand.append(lowerBand[i-1])

df['upperband'] = upperBand
df['lowerband'] = lowerBand
df.drop(['basicUpperband', 'basicLowerband'], axis=1, inplace=True)
return df

```

So this “*supertrend*” function takes the dataframe we got from the last function. It also needs one other thing — the “*atr_multiplier*” number.

It calculates the Average True Range or ATR. That tells us how much the price has been moving up and down each day or any period configured, so we know how “volatile” or jumpy the market is. It uses the ATR and the multiplier number to calculate the basic upper band and basic lower band each row.

Now it starts looking at each row, except the very first one. For the upper band — if the basic upper band is lower than the final upper band from the row before, OR if the close price is above the final upper band from the row before, it adds the basic upper band value to the final upper band list.

It does the same thing for the lower band, just checking if the basic lower band is higher than the final bottom band from the row before, OR if the close price is below the final lower before.

After checking every row, it drops the columns we don't need anymore and returns the dataframe. These final bands (*df['upperband']* and *df['lowerband']*) and the closing price will tell us the signals for when to buy or sell.

Step 5: Generate the signals

```
def generate_signals(df):  
    # Intiate a signals list  
    signals = [0]  
  
    # Loop through the dataframe
```

```

for i in range(1 , len(df)):
    if df['close'][i] > df['upperband'][i]:
        signals.append(1)
    elif df['close'][i] < df['lowerband'][i]:
        signals.append(-1)
    else:
        signals.append(signals[i-1])

# Add the signals list as a new column in the dataframe
df['signals'] = signals
df['signals'] = df["signals"].shift(1) #Remove look ahead bias
return df

```

The function “*generate_signals*” starts by making a list called “signals” and setting the first signal to 0. Then it looks through the whole dataframe row by row to find buy and sell signals.

A buy signal happens when the closing price for that row goes above the upper band value. A sell signal happens when the closing price drops below the lower band value.

If the current signal is the same as the previous one, we keep the position the same as the last row.

Each time it looks at a new row, it updates the signals list with the new

signal for that row. Then it adds the whole signals list as a new column in the dataframe called “*df['signals']*”.

It removes the look ahead bias to simulate taking positions in a similar way you would in a live trading environment after you have seen the signal. That way we don't get signals using information from the future, thus the backtested results are more realistic and reliable. The functions then returns the “*df*”— which will be used to create positions.

Step 6: Generate the positions

```
def create_positions(df):  
    # We need to shut off (np.nan) data points in the upperband where the sig  
    df['upperband'][df['signals'] == 1] = np.nan  
    # We need to shut off (np.nan) data points in the lowerband where the sig  
    df['lowerband'][df['signals'] == -1] = np.nan  
  
    # Create a positions list  
    buy_positions = [np.nan]  
    sell_positions = [np.nan]  
  
    # Loop through the dataframe  
    for i in range(1, len(df)):  
        # If the current signal is a 1 (Buy) & the it's not equal to the prev  
        # Then that is a trend reversal, so we BUY at that current market pri
```

```

# We take note of the upperband value
if df['signals'][i] == 1 and df['signals'][i] != df['signals'][i-1]:
    buy_positions.append(df['close'][i])
    sell_positions.append(np.nan)
# If the current signal is a -1 (Sell) & the it's not equal to the pr
# Then that is a trend reversal, so we SELL at that current market pr
elif df['signals'][i] == -1 and df['signals'][i] != df['signals'][i-1]:
    sell_positions.append(df['close'][i])
    buy_positions.append(np.nan)
else:
    buy_positions.append(np.nan)
    sell_positions.append(np.nan)

# Add the positions list as a new column in the dataframe
df['buy_positions'] = buy_positions
df['sell_positions'] = sell_positions
return df

```

The function starts off by getting rid of some numbers in the upper and lower bands that we don't need to see. Remember how those upper and lower bands were calculated before? Well, sometimes the bands would keep showing the closing price values even when the signal didn't say to buy or sell there. That could get confusing later on when we want to just look at the bands where we actually have an open position (trend). So those first two lines in the function are just cleaning things up for us. It's putting "NaN" in the spots of the upper band where the signal isn't "1", and in the lower band where the signal isn't "-1". Especially when we want

to plot the bands — you don't want to plot the entire lines, only the spots where the trend is active. So you know — “NaN” is a special number that means “no value”. This will make more sense as we move on.

We go ahead and initialize two empty lists — one for our buy prices and one for our sell prices. The function then loops through the dataframe looking at each row, except the very first one.

So if the signal is a “1” (meaning buy) and it's different than the signal before, that's a reversal so we make a buy. It adds the close price for that row to the buy list. In a similar fashion, if the signal is a “-1” (sell) and different than before, that's a sell. It adds the close price to the sell list. If nothing changed, it just leaves those spots empty(“NaN”) in the lists.

After checking every row, it adds the final buy and sell lists as new columns to the dataframe. Now we have a clear record of exactly when we would have bought and sold based on the signals.

Step 7: Visualize the data

```
def plot_data(df, symbol):
    # Define lowerband line plot
    lowerband_line = mpf.make_addplot(df['lowerband'], label= "lowerband", co
    # Define upperband line plot
    upperband_line = mpf.make_addplot(df['upperband'], label= "upperband", co
    # Define buy and sell markers
    buy_position_makers = mpf.make_addplot(df['buy_positions'], type='scatter
    sell_position_makers = mpf.make_addplot(df['sell_positions'], type='scatt
    # A list of all addplots(apd)
    apd = [lowerband_line, upperband_line, buy_position_makers, sell_position
    # Create fill plots
    lowerband_fill = dict(y1=df['close'].values, y2=df['lowerband'].values, p
    upperband_fill = dict(y1=df['close'].values, y2=df['upperband'].values, p
    fills = [lowerband_fill, upperband_fill]
    # Plot the data
    mpf.plot(df, addplot=apd, type='candle', volume=True, style='charles', xr
```

Now we want to visualize the data, the function “*plot_data*”, makes lines on the chart to show the lower and upper bands we calculated before. The lower band will be green and the upper will be red.

It’s also making markers to show exactly when we bought and sold — little up arrows for buys and down arrows for sells. These will be a little big and colorful so they kind of stand out. Then it groups all those lines and markers together so we can add them to the chart. Then, it goes ahead to

add shaded areas between the close price and the bands. This will make it really clear to visualize the up and down trends.

It plots the candlesticks for the ohlv (open, high, low, close), sets the title and legends, and then shows the plot. Now we will easily see how the strategy would have played out over time.

Step 8: Calculate the strategy performance

```
def strategy_performance(strategy_df, capital=100, leverage=1):
    # Initialize the performance variables
    cumulative_balance = capital
    investment = capital
    pl = 0
    max_drawdown = 0
    max_drawdown_percentage = 0

    # Lists to store intermediate values for calculating metrics
    balance_list = [capital]
    pnl_list = [0]
    investment_list = [capital]
    peak_balance = capital

    # Loop from the second row (index 1) of the DataFrame
    for index in range(1, len(strategy_df)):
        row = strategy_df.iloc[index]
```

```

# Calculate P/L for each trade signal
if row['signals'] == 1:
    pl = ((row['close'] - row['open']) / row['open']) * \
        investment * leverage
elif row['signals'] == -1:
    pl = ((row['open'] - row['close']) / row['close']) * \
        investment * leverage
else:
    pl = 0

# Update the investment if there is a signal reversal
if row['signals'] != strategy_df.iloc[index - 1]['signals']:
    investment = cumulative_balance

# Calculate the new balance based on P/L and leverage
cumulative_balance += pl

# Update the investment list
investment_list.append(investment)

# Calculate the cumulative balance and add it to the DataFrame
balance_list.append(cumulative_balance)

# Calculate the overall P/L and add it to the DataFrame
pnl_list.append(pl)

# Calculate max drawdown
drawdown = cumulative_balance - peak_balance
if drawdown < max_drawdown:
    max_drawdown = drawdown
    max_drawdown_percentage = (max_drawdown / peak_balance) * 100

# Update the peak balance
if cumulative_balance > peak_balance:
    peak_balance = cumulative_balance

```

```

# Add new columns to the DataFrame
strategy_df['investment'] = investment_list
strategy_df['cumulative_balance'] = balance_list
strategy_df['pl'] = pnl_list
strategy_df['cumPL'] = strategy_df['pl'].cumsum()

# Calculate other performance metrics (replace with your calculations)
overall_pl_percentage = (
    strategy_df['cumulative_balance'].iloc[-1] - capital) * 100 / capital
overall_pl = strategy_df['cumulative_balance'].iloc[-1] - capital
min_balance = min(strategy_df['cumulative_balance'])
max_balance = max(strategy_df['cumulative_balance'])

# Print the performance metrics
print("Overall P/L: {:.2f}%".format(overall_pl_percentage))
print("Overall P/L: {:.2f}".format(overall_pl))
print("Min balance: {:.2f}".format(min_balance))
print("Max balance: {:.2f}".format(max_balance))
print("Maximum Drawdown: {:.2f}".format(max_drawdown))
print("Maximum Drawdown %: {:.2f}%".format(max_drawdown_percentage))

# Return the Strategy DataFrame
return strategy_df

```

We need to check on some metrics to see how well our trading strategy does. That is what this function “*strategy_performance*” does. It figures out first of all what our money would be worth if we just held onto it — that’s the benchmark.

Now it sets up some empty lists and variables to track things like the balance, profits and losses (P&L), and how much money we invest during each position. It then looks at each row in our strategy dataframe, one by one. For each signal, it calculates the P&L by seeing how much the asset price changed between when we bought and sold, and then updates our balance, investment amount, and adds the new numbers to the lists. It also tracks other metrics like the highest and lowest balance we ever had, overall profit, maximum drawdown etc. These, it prints out. Note that this functions also compounds the returns, i.e the investment is the available balance from even all the closed positions.

Step 9: Visualize the strategy performance

```
def plot_performance_curve(strategy_df):  
    # Plot the strategy performace curve  
    plt.plot(strategy_df['cumulative_balance'], label='Strategy')  
    plt.title('Performance Curve')  
    plt.xlabel('Date')  
    plt.ylabel('Balance')  
    plt.xticks(rotation=70)  
    plt.legend()  
    plt.show()
```

We plot the strategy and the benchmark returns to compare how well the strategy would have performed.

Step 10: Run the Script

```
if __name__ == '__main__':  
    # Initialize data fetch parameters  
    symbol = "BTC/USDT"  
    start_date = "2024-1-1"  
    interval = '4h'  
    exchange = ccxt.binance()  
  
    # Fetch historical OHLC data for ETH/USDT  
    data = fetch_asset_data(symbol=symbol, start_date=start_date, interval=in  
  
    volatility = 3  
  
    # Apply supertrend formula  
    supertrend_data = supertrend(df=data, atr_multiplier=volatility)  
  
    # Generate the Signals  
    supertrend_positions = generate_signals(supertrend_data)  
  
    # Generate the Positions  
    supertrend_positions = create_positions(supertrend_positions)  
  
    # Calculate performance  
    supertrend_df = strategy_performance(supertrend_positions, capital=100, l  
    print(supertrend_df)
```

```
# Plot data
plot_data(supertrend_positions, symbol=symbol)

# Plot the performance curve
plot_performance_curve(supertrend_df)
```

Set up the variables under the “*if __name__ == '__main__':*” constructor, and then get to the terminal point, it to the directory containing file and then run the script.

My file was *supertrend.py* so, you run it by running “*python supertrend.py*” in your terminal.

Here is the resulting dataframe:

And these were the calculated performance metrics, Note that past performance does not guarantee future results. So use this strategy with caution. And of course, nothing in this article is investment advice. It's all for educational purposes.

As you can see for a period of 3-months, with an investment of **\$100** and no leverage, BTC/USDT would have made a **40%** return on investment. Feel free to calculate other metrics like Sortino ratio, alpha e.t.c

And this was the strategy plot (it is a little zoomed out so some data points may be cut off)

And lastly here is the resulting plot for the performance curve:

Limitations of the strategy

Parameter Sensitivity: The supertrend indicator can be sensitive to its parameters such as the period, and the atr volatility multiplier. Different trading pairs can move around a lot in a given period, but be pretty quiet on others. So what worked well for one pair might not work as good for a different one. You could tweak the volatility parameters to suit the trading pair.

Volatility Differences: The SuperTrend indicator relies heavily on the ATR to capture trends, but if a trading pair is so volatile or experiences frequent sharp price fluctuations, it may produce false signals or fail to capture meaningful trends. Therefore the supertrend indicator may not be suitable for highly volatile assets.

To wrap up, the super trend strategy is a powerful tool that can assist investors in making some well-informed decisions regarding their trading activities. It assists in identifying the current trends and reversals in the market, enabling traders to execute timely entry and exit signals. When implemented properly, this strategy can result in significantly higher returns, making it an invaluable resource.

Here is the full code on my [Github](#)

Stackademic 🎓

Thank you for reading until the end. Before you go:

- Please consider **clapping** and **following** the writer! 🙌
- Follow us [X](#) | [LinkedIn](#) | [YouTube](#) | [Discord](#)

- Visit our other platforms: [In Plain English](#) | [CoFeed](#) | [Venture](#) | [Cubed](#)
- More content at [Stackademic.com](#)

Algorithmic Trading

Data Science

Trading

Software Development

Python



Published in Stackademic

85K followers · Last published 1 hour ago

Follow

Stackademic is a learning hub for programmers, devs, coders, and engineers. Our goal is to democratize free coding education for the world.



Written by Tradie Capital

311 followers · 9 following

Follow

An AI-Powered Trading analytics platform. Building tools that help traders analyze markets faster and make better decisions. www.tradiecapital.com

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Store](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)